

# Text Preprocessing & Cleaning

---

## 1. What is Text Preprocessing?

Imagine you're cooking 🍳

You don't throw raw vegetables directly into the dish, right?

You:

- Wash them
- Cut them
- Remove unwanted parts

👉 **Text preprocessing is the same idea**

Raw text = messy ingredients

Preprocessed text = clean, ready-to-use data

💡 In simple words:

Text preprocessing = **cleaning and preparing text so machines can understand it better**

---

## 2. Why Do We Need It? (Problem → Solution)

### **Problem:**

Real-world text is messy:

"OMG!!! I loooove this product 🥰🥰 !!! It's AMAZING!!!"

Issues:

- Extra punctuation
- Repeated letters
- Emojis
- Mixed casing
- Noise

👉 **Machines get confused**

---

## ✅ **Solution:**

Clean and standardize the text:

"love product amazing"

👉 Now the model understands clearly

---

## 🧩 **3. Core Components**

We'll go step by step 👉

---

### ◆ **1. Tokenization**

#### 💡 **What is it?**

Breaking text into smaller pieces (tokens)

#### 📦 **Example:**

"I love NLP"

→ ["I", "love", "NLP"]

#### 💡 **Analogy:**

Like breaking a sentence into LEGO blocks 🧱

---

### ◆ **2. Normalization**

👉 Making text consistent

---

## ✂️ **Stemming (Rough Cutting)**

#### 💡 **Idea:**

Cut words to root form (not always correct)

playing → play  
studies → studi ❌

👉 Fast but sometimes inaccurate

---

## **Lemmatization (Smart Reduction)**

### **Idea:**

Convert word → meaningful root using dictionary

studies → study ✅  
better → good ✅

👉 Slower but more accurate

---

## **Stemming vs Lemmatization**

| Feature  | Stemming         | Lemmatization |
|----------|------------------|---------------|
| Accuracy | Low              | High          |
| Speed    | Fast             | Slower        |
| Output   | May be incorrect | Correct word  |

---

## ◆ **3. Stop Word Removal**

### **What are stopwords?**

Common words with little meaning:

- is, the, am, are, was
- 

### **Example:**

"I am going to the store"  
→ ["going", "store"]

👉 Removes noise, keeps important words

---

## ◆ 4. Regular Expressions (Regex)

### 💡 What is Regex?

A **pattern-matching tool** for text cleaning

---

### 📦 Examples:

**Remove numbers:**

"I have 2 apples" → "I have apples"

**Remove special characters:**

"Hello!!!" → "Hello"

---

### 💡 Analogy:

Regex = **search & replace superpower** 🔍

---

## ◆ 5. Handling Noisy Data

Real-world text contains:

- Emojis 😊
  - Punctuation !!!
  - Contractions ("don't")
  - Special characters (#, @)
- 

### 🔧 Cleaning Steps:

#### 1. Remove punctuation

"Hello!!!" → "Hello"

---

#### 2. Remove special characters

"@user #AI" → "AI"

---

### 3. Expand contractions

"don't" → "do not"

"it's" → "it is"

---

### 4. Lowercasing

"AI is GREAT" → "ai is great"

---

## 4. How It All Works Together

### Intuitive Flow:

Raw Text  
↓  
Lowercase  
↓  
Remove Noise  
↓  
Tokenize  
↓  
Remove Stopwords  
↓  
Stemming/Lemmatization  
↓  
Clean Text Output

---

### Full Example:

#### Input:

"I don't like this product!!! 🤔"

#### Step-by-step:

1. Lowercase  
→ "i don't like this product!!! 🤔"
2. Expand contraction  
→ "i do not like this product"
3. Remove punctuation/emojis  
→ "i do not like this product"
4. Tokenize  
→ ["i", "do", "not", "like", "this", "product"]

5. Remove stopwords  
→ ["not", "like", "product"]
  6. Lemmatize  
→ ["not", "like", "product"]
- 

✅ **Final Output:**

["not", "like", "product"]

---

## 🔥 5. Why This is CRITICAL (For You)

As a **Data Engineer + AI Engineer**, preprocessing is:

- The **foundation of NLP pipelines**
- Used before every model
- Critical for:
  - Chatbots
  - RAG systems
  - Log analysis
  - Healthcare text pipelines

💡 Bad preprocessing = Bad model (100% true in interviews)

---

## 🧠 Final Mental Model

✂️ Text Preprocessing = Cleaning + Structuring + Simplifying Text for Machines

---

## 🌍 3. Real-World Use Cases

---

### 🏥 Healthcare (Your Domain)

- Clean doctor notes
  - Extract symptoms from text
-

## Customer Support

- Clean chat logs before analysis
- 

## E-commerce

- Preprocess product reviews
- 

## Data Engineering Pipeline (Your Strong Area)

S3 → Glue → Preprocessing (this code) → Feature Engineering → Model → Snowflake

---

## 4. Mini Project

### Project: "Text Cleaning API"

---

#### Goal:

Build an API that cleans input text

---

#### Steps:

1. Create preprocessing function
  2. Wrap it in FastAPI or Flask
  3. Input → Raw text
  4. Output → Clean tokens
- 

#### Example Input:

"I can't believe this!!! 🤯"

#### Output:

["believe"]

---

## Tools:

- Python
  - NLTK
  - Regex
  - FastAPI (optional)
- 

# 5. Common Mistakes (Very Important)

---

## 1. Over-cleaning

 Removing important words like "not"

- "not good" → "good"  (wrong meaning)
- 

## 2. Blindly removing stopwords

 Sometimes "not", "no" are important

---

## 3. Using stemming in production

 Can distort meaning

---

## 4. Ignoring contractions

 "don't" ≠ "do"

---

## 5. Wrong regex

 Can accidentally remove useful data

---



# Final Practical Understanding

👉 Full Pipeline:

```
graph TD; A[Raw Text] --> B[Lowercase]; B --> C[Expand Contractions]; C --> D[Remove Noise (Regex)]; D --> E[Tokenize]; E --> F[Remove Stopwords]; F --> G[Stem / Lemmatize]; G --> H[Clean Tokens];
```

Raw Text  
↓  
Lowercase  
↓  
Expand Contractions  
↓  
Remove Noise (Regex)  
↓  
Tokenize  
↓  
Remove Stopwords  
↓  
Stem / Lemmatize  
↓  
Clean Tokens

---